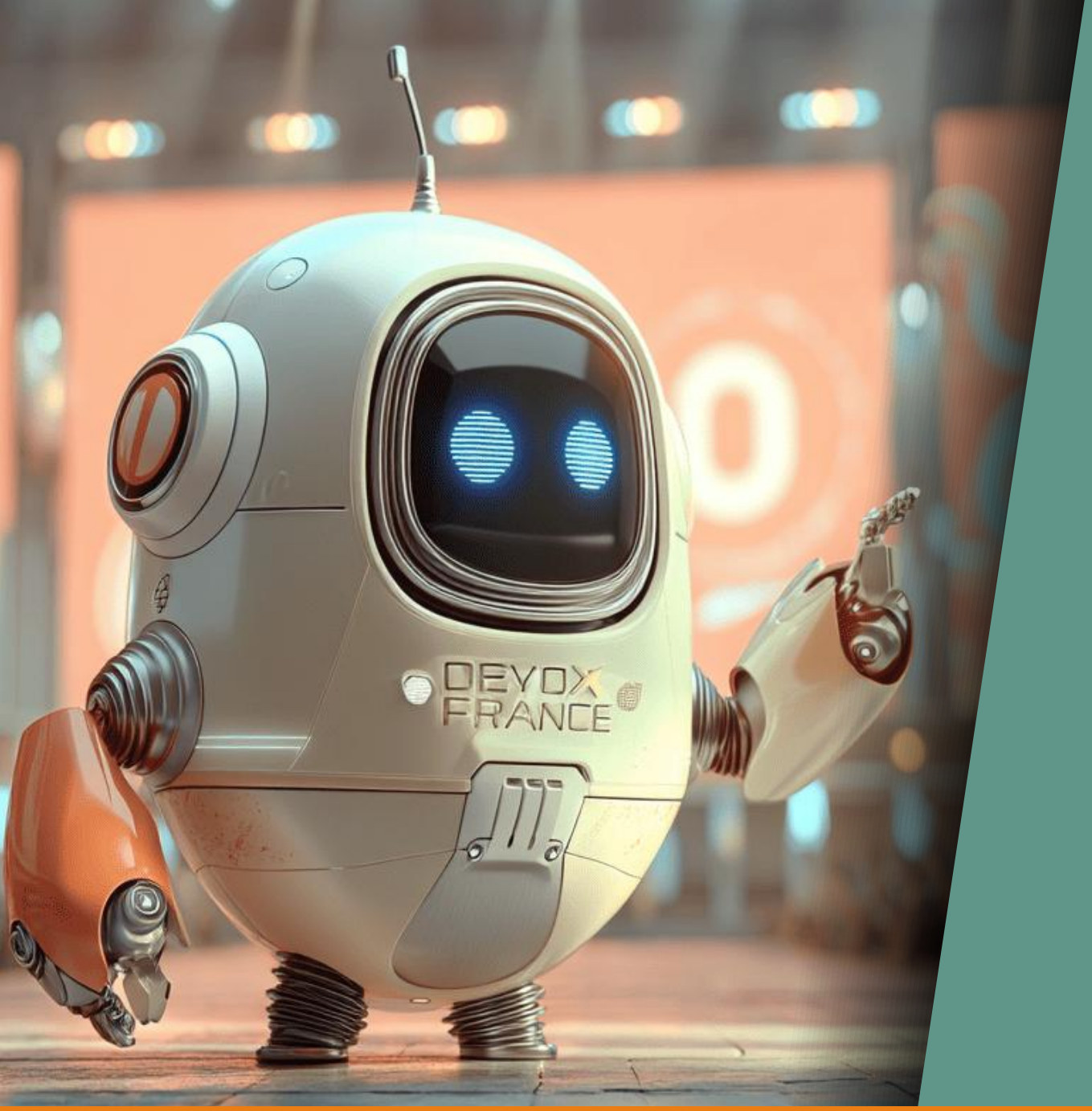
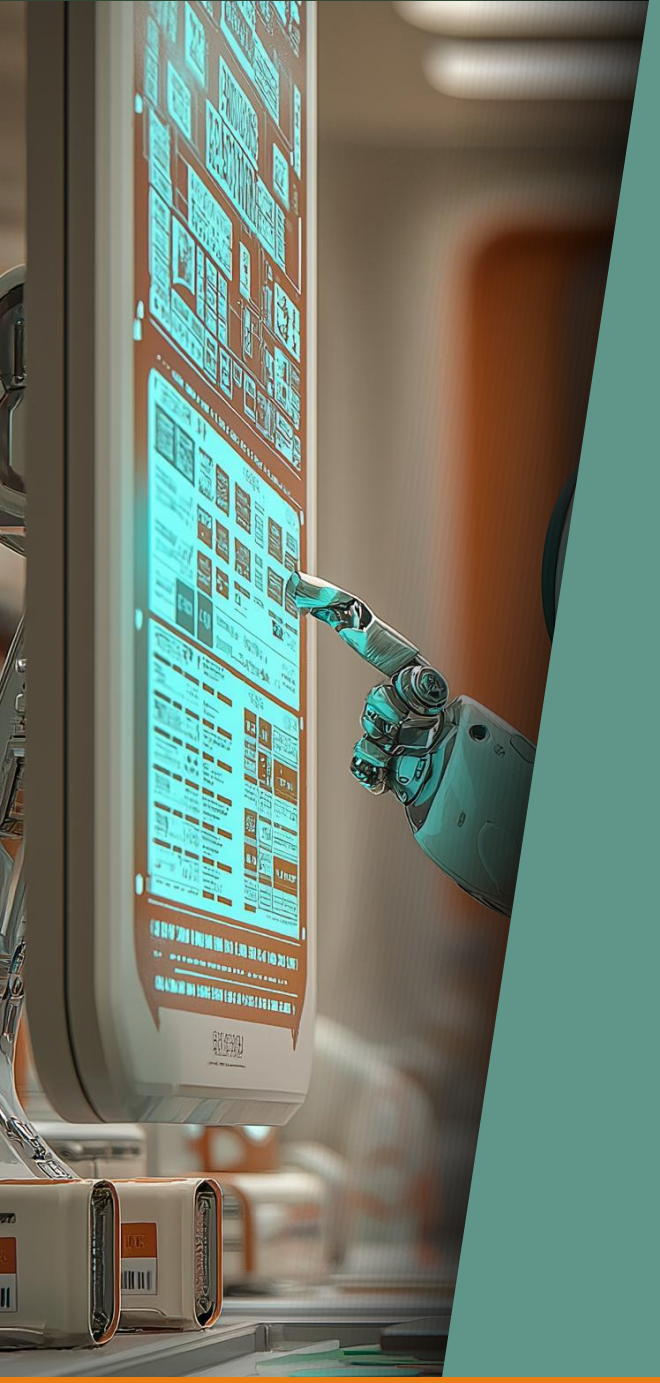


DEVOXX FRANCE 2026



LE GUIDE DU TESTEUR

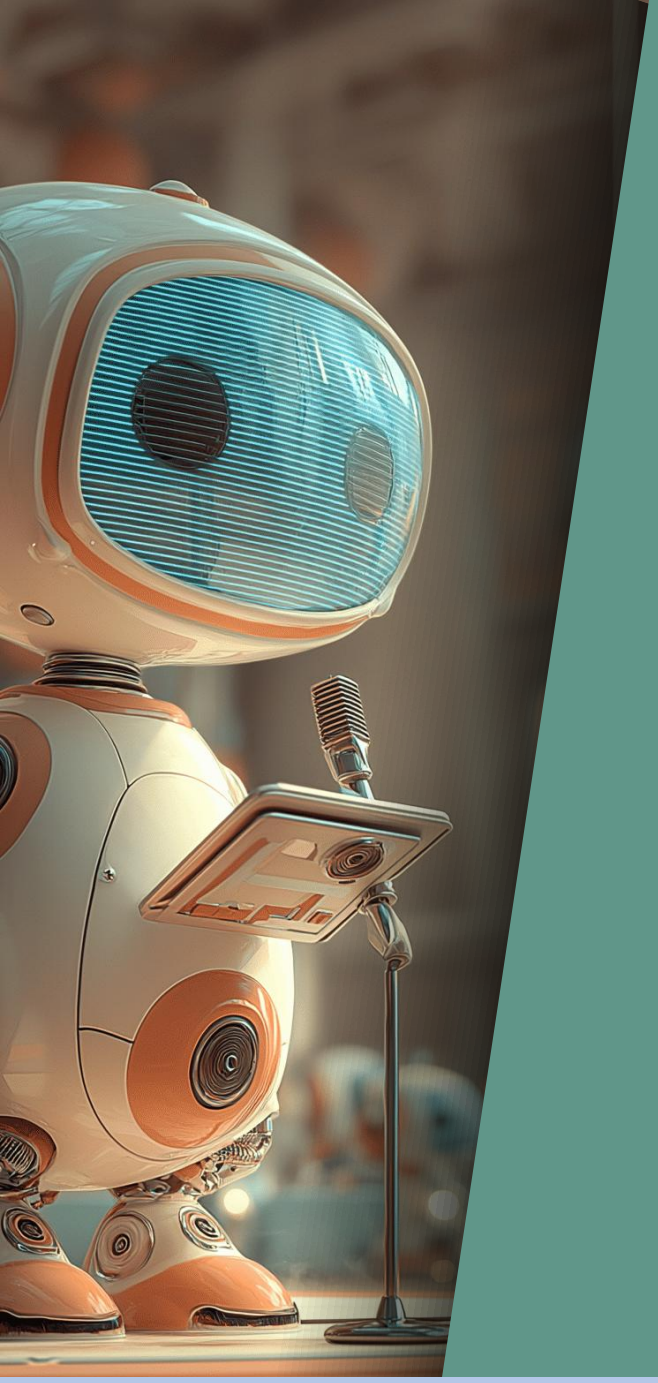
6 MAUVAISES HABITUDES À BRISER ABSOLUMENT

Chadi EL MASRI

22 Avril 2026

LES PILIERS D'UN BON TEST

- Lisibilité immédiate
- Utilité réelle
- Déterminisme total
- Échec explicite



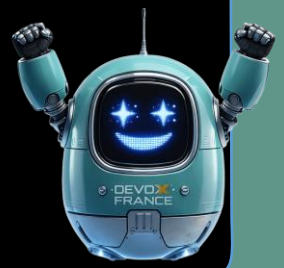
Mauvaise Habitude #1

LES TESTS ILLISIBLES

```java
@Test
void complexOrderProcessing_scenarioTest_IsImpossibleToFollow() {
    CustomerService service = new CustomerService();
    int customerId = 5001;
    double initialAmount = 1200.0;
    String result = service.processOrder(customerId, initialAmount, true, "FREE_SHIP");
    assertEquals("PENDING_APPROVAL", result);
    assertEquals(5, service.getAuditLog().size());
    assertEquals("Start processing for 5001", service.getAuditLog().get(0));
    assertTrue(service.getAuditLog().get(1).contains("VIP discount"));
    assertEquals("Shipping waived", service.getAuditLog().get(2));
    String secondResult = service.processOrder(10, 50.0, false, "");
    assertEquals("REJECTED_INVALID_INPUT", secondResult);
    assertEquals(6, service.getAuditLog().size());
    assertFalse(service.getAuditLog().contains("Order complete"));
    CustomerService newService = new CustomerService();
    String thirdResult = newService.processOrder(200, 400.0, false, "NOCode");
    assertEquals("COMPLETED", thirdResult);
    assertEquals(2, newService.getAuditLog().size());
}
```

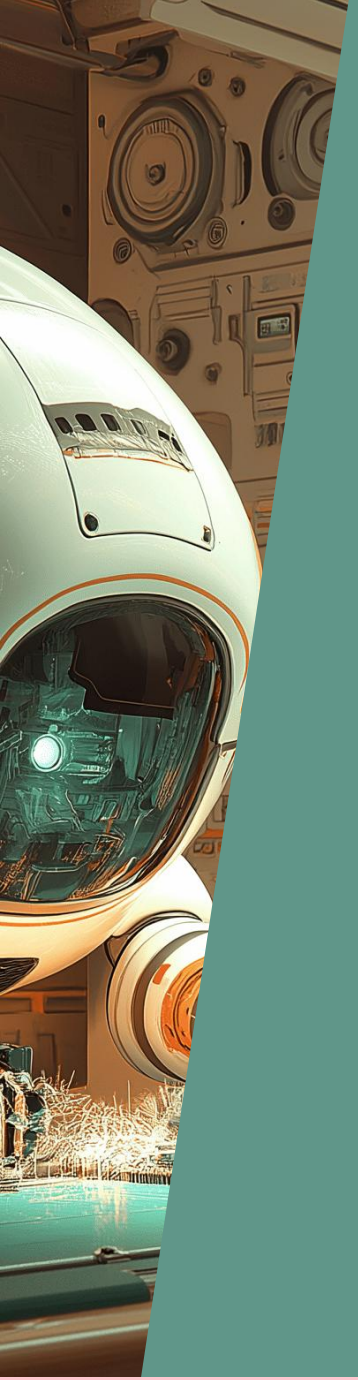
GIVEN, WHEN, THEN

```
@Test
void highValueVipOrderWithFreeShipping_shouldRequireApproval() {
    // GIVEN - Un client VIP avec une commande de grande valeur et livraison gratuite
    CustomerService service = new CustomerService();
    int customerId = 5001;
    double initialAmount = 1200.0;

    // WHEN - La commande est traitée
    String result = service.processOrder(customerId, initialAmount, true, "FREE_SHIP");

    // THEN - La commande nécessite une approbation
    assertEquals("PENDING_APPROVAL", result);
    assertEquals(5, service.getAuditLog().size());
    assertEquals("Start processing for 5001", service.getAuditLog().get(0));
    assertTrue(service.getAuditLog().get(1).contains("VIP discount"));
    assertEquals("Shipping waived", service.getAuditLog().get(2));
}
```



Mauvaise Habitude #2

L'OBSESSION DE LA MÉTRIQUE DE COUVERTURE



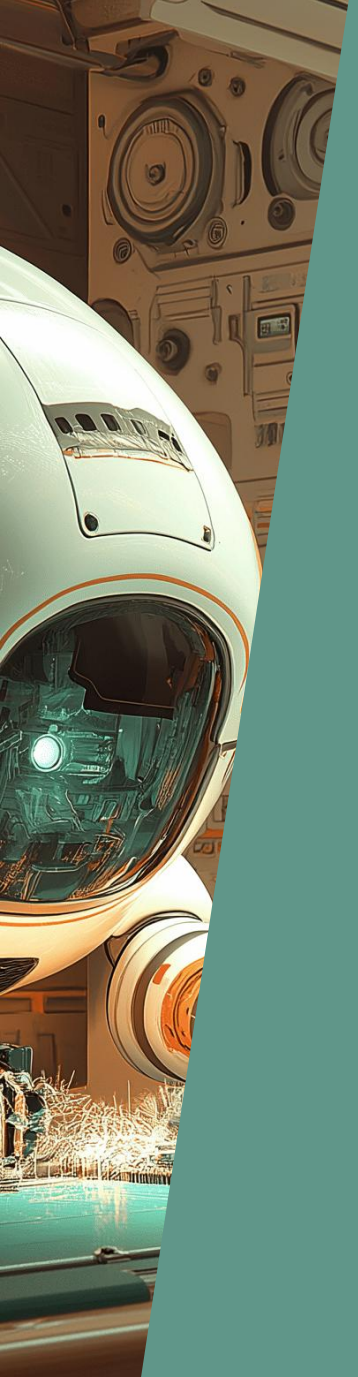
Codecov Report

❌ Patch coverage is `0%` with `9 lines` in your changes missing coverage. Please review.
✅ Project coverage is 63.15%. Comparing base (`4aabf52`) to head (`5c8d413`).

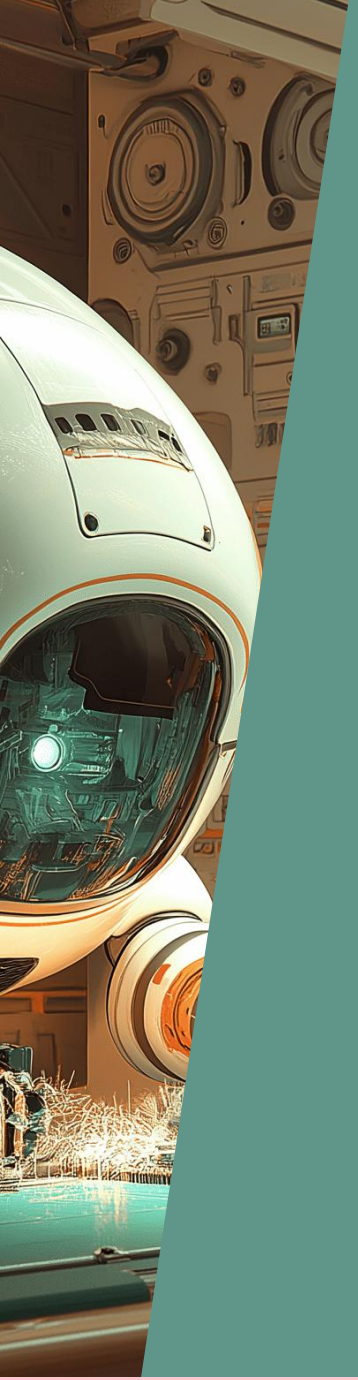
Files with missing lines	Patch %	Lines
src/main/java/io/chadi/User.java	0.00%	6 Missing ⚠️
src/main/java/io/chadi/Displayer.java	0.00%	3 Missing ⚠️

▸ Additional details and impacted files

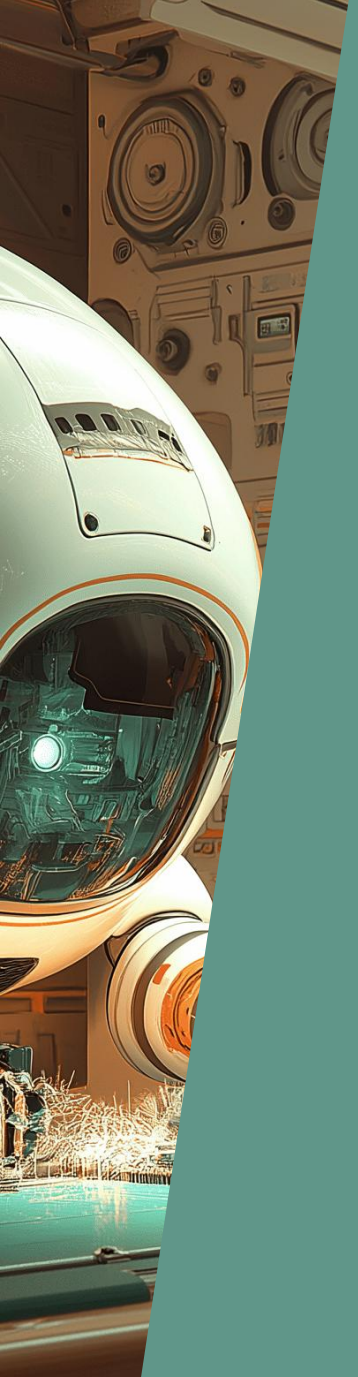
☂️ View full report in Codecov by Sentry.
📢 Have feedback on the report? Share it here.



```java
public class User {
    private long id;
    private String username;

    public long getId() { return id; }
    public void setId(long id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) {
        this.username = username;
    }
}
```

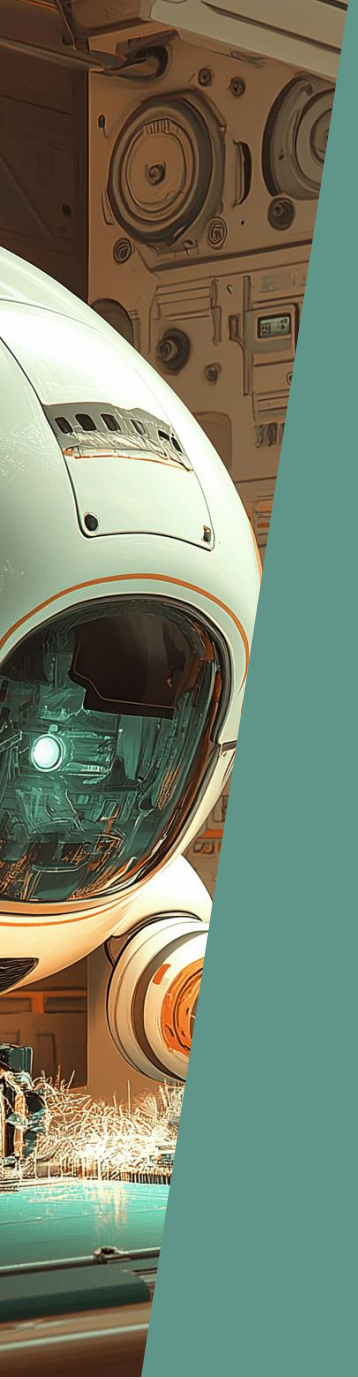


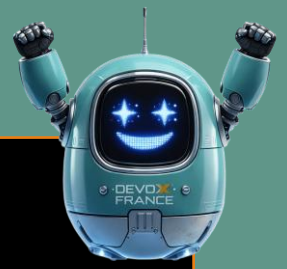
```
class UserTest {
    @Test
    void testGetAndSetId() {
        User user = new User();
        user.setId(123L);
        assertEquals(123L, user.getId());
    }

    @Test
    void testGetAndSetUsername() {
        User user = new User();
        user.setUsername("testuser");
        assertEquals("testuser", user.getUsername());
    }
}
```



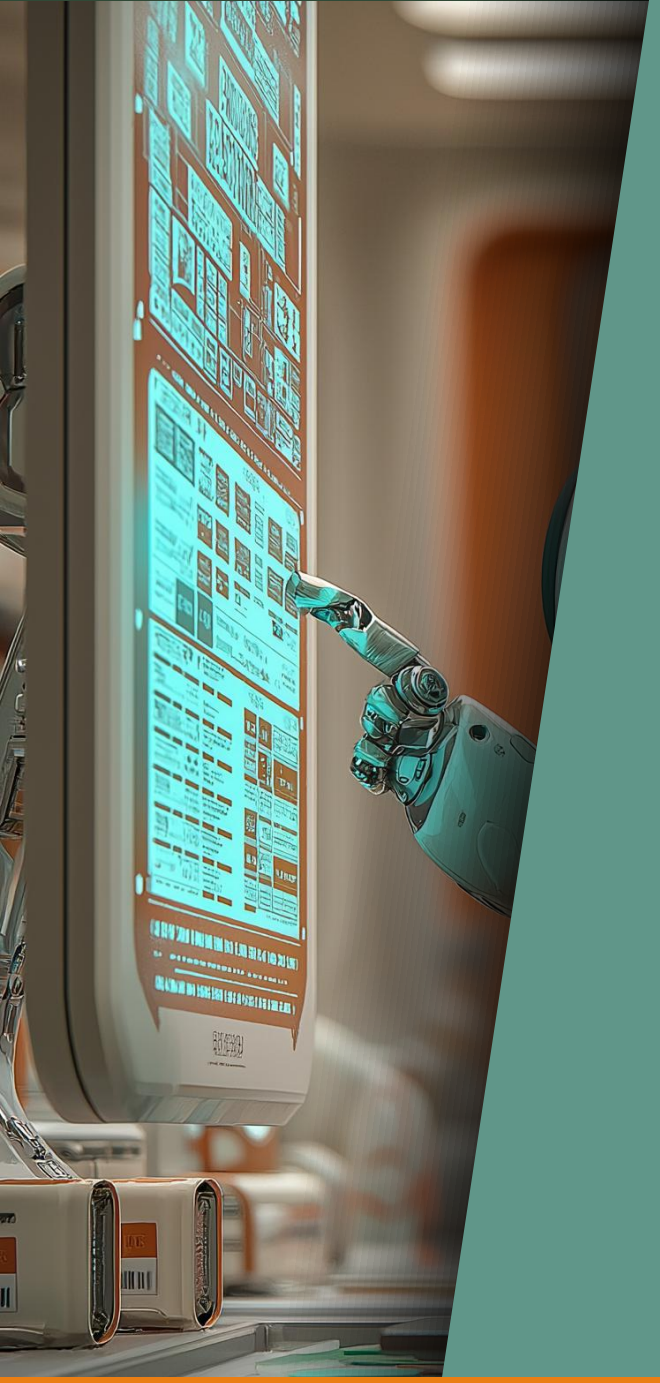
LE PARADOXE DE LA COUVERTURE

- La couverture de code est un **indicateur**, pas un **objectif**.
- Elle montre si le code est **exécuté**, pas s'il est **utile** ou **correct**.
- Il vaut mieux tester la **fonctionnalité**.






```
public record User(long id, String username) {
    // Le compilateur gère les propriétés et méthodes.
}


class DisplayerTest {
    @Test
    void test_display() {
        var user = new User(1, "Bob");

        var onDisplay = Displayer.display(user);

        assertEquals("Bob/1", onDisplay);
    }
}
```



Mauvaise Habitude #3

LA SURUTILISATION DES MOCKS

SURUTILISATION DES MOCKS

- Phénomène souvent présent lorsqu’on teste des classes de type **Orchestrateur**
  - Service externes
  - Bases de données
- Le mocking est nécessaire, mais son excès rend le test inutile
  - On teste les mocks, plus le vrai code

```java
// 6 Collaborateurs mockés : Le test devient un miroir de l'implémentation
private final Validator validator = mock(Validator.class);
private final PriceCalculator calculator = mock(PriceCalculator.class);
private final StockService stock = mock(StockService.class);
private final LoyaltyService loyalty = mock(LoyaltyService.class);
private final NotificationService notifier = mock(NotificationService.class);
private final OrderRepository repo = mock(OrderRepository.class);

private final OrderService service = new OrderService(
    validator, calculator, stock, loyalty, notifier, repo);

@Test
void testProcessOrder_TheMockingNightmare() {
    // GIVEN: On doit tout scripter, même les détails insignifiants
    Order order = new Order("SKU-123", 100.0, "user-42");
    when(validator.isValid(order)).thenReturn(true);
    when(calculator.calculateFinalPrice(100.0)).thenReturn(90.0);
    when(stock.isAvailable("SKU-123")).thenReturn(true);

    // WHEN
    service.process(order);

    // THEN: On vérifie chaque interaction technique
    // Si on change l'ordre des appels ou un détail interne, tout explose.
    verify(validator).isValid(order);
    verify(stock).isAvailable("SKU-123");
    verify(calculator).calculateFinalPrice(100.0);
    verify(loyalty).addPoints("user-42", 9);
    verify(repo).save(order);
    verify(notifier).sendConfirmation(order);
}
```

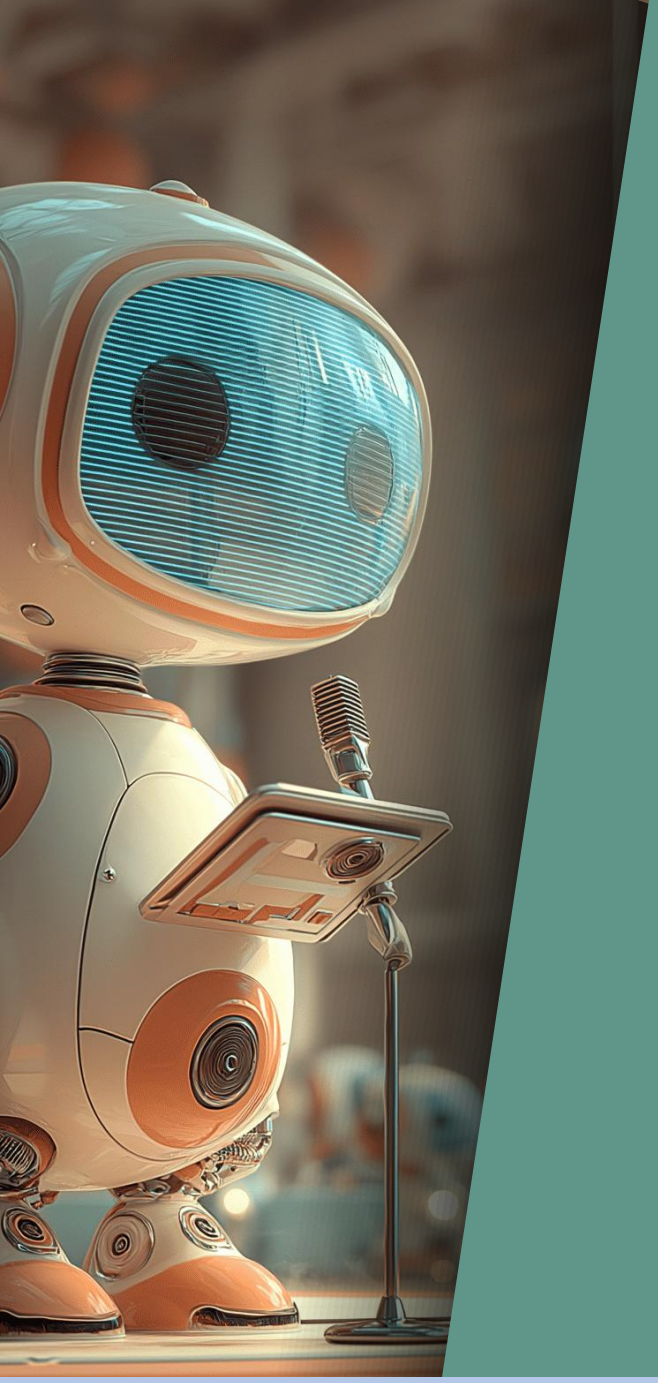
SURUTILISATION DES MOCKS

Illusion de securité

- Au moindre changement d’implémentation, si le mock n’est pas adapté, le test casse.
- Ce test crée un faux sentiment de sécurité : il peut très bien passer en local, mais ne pas du tout représenter la production.

QUELLES SOLUTIONS?

- Toujours utiliser la **vraie implémentation** lorsque c'est possible
- Monter dans la pyramide des tests :
  - IT avec **Testcontainers**
  - Tests **E2E**
- S'il y a des algorithmes dans ces classes orchestrateurs, ils pourraient être testés séparément dans des classes utilitaires



Mauvaise Habitude #4

LES ASSERTIONS PEU LISIBLES DE COLLECTIONS

```java
@Test
void checkUserListProperties_usingVanillaJUnit() {
    List<User> users = service.findAll();

    assertEquals(2, users.size());
    assertTrue(users.stream().anyMatch(u ->
            u.name().equals("Alice") && u.role().equals("ADMIN")));
    assertTrue(users.stream().anyMatch(u ->
            u.name().equals("Bob") && u.role().equals("VIEWER")));
}
```

```
org.opentest4j.AssertionFailedError:
Expected :true
Actual   :false
<Click to see difference>
```

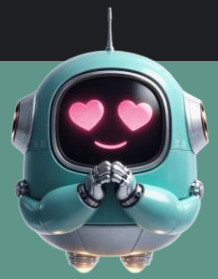
```java
@Test
void checkUserListProperties_usingVanillaJUnit_withExplicitMessages() {
    List<User> users = service.findAll();

    assertEquals(2, users.size());
    assertTrue(users.stream().anyMatch(u ->
                    u.name().equals("Alice") && u.role().equals("ADMIN")),
            "Expecting Alice/ADMIN, not found");
    assertTrue(users.stream().anyMatch(u ->
                    u.name().equals("Bob") && u.role().equals("VIEWER")),
            "Expecting Bob/VIEWER, not found");
}
```

```
org.opentest4j.AssertionFailedError: Expecting Bob/VIEWER, not found ⟹
Expected :true
Actual   :false
<Click to see difference>
```

AssertJ

```
@Test
void checkUserListProperties_usingAssertJ() {
    List<User> allUsers = service.findAll();

    assertThat(allUsers)
        .hasSize(2)
        .extracting(User::name, User::role)
        .containsInAnyOrder(
              tuple("Alice", "ADMIN"),
              tuple("Bob", "VIEWER")
        );
}
```

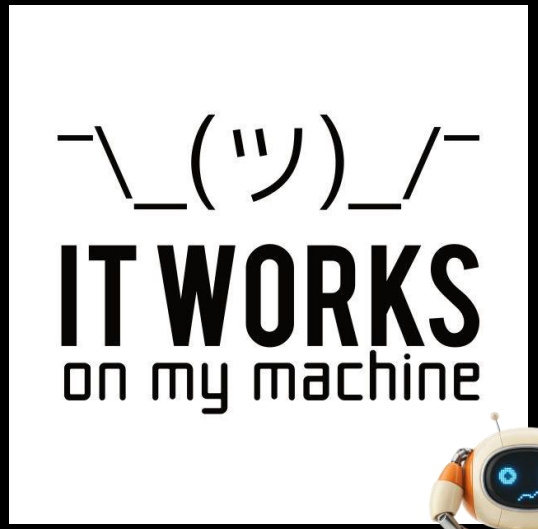
```
java.lang.AssertionError:
Expecting ArrayList:
  [("Alice", "ADMIN"), ("Bob", "ADMIN")]
to contain:
  [("Alice", "ADMIN"), ("Bob", "VIEWER")]
but could not find the following element(s):
  [("Bob", "VIEWER")]
```



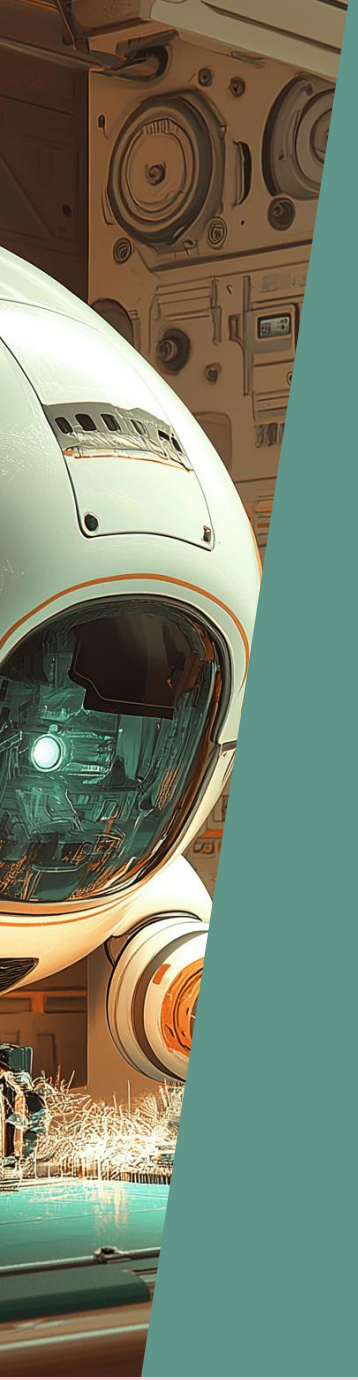
Mauvaise Habitude #5

LE THREAD.SLEEP() MAL UTILISÉ

```java
@Test
void shouldProcessAsynchronously_Flaky() throws Exception {
    // GIVEN
    var processor = new AsyncProcessor();

    // WHEN
    processor.processAsync();

    // THEN: On espère que 150ms suffisent…
    Thread.sleep(150);
    assertTrue(processor.hasSucceeded(),
            "Processing should have succeeded.");
}
```

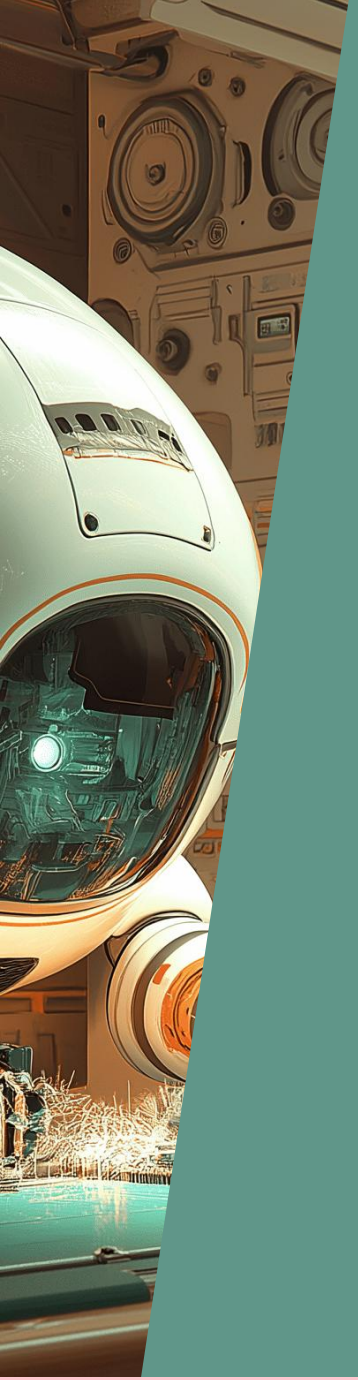


```
org.opentest4j.AssertionFailedError: Processing should have succeeded. ⟹
Expected :true
Actual   :false
<Click to see difference>
```

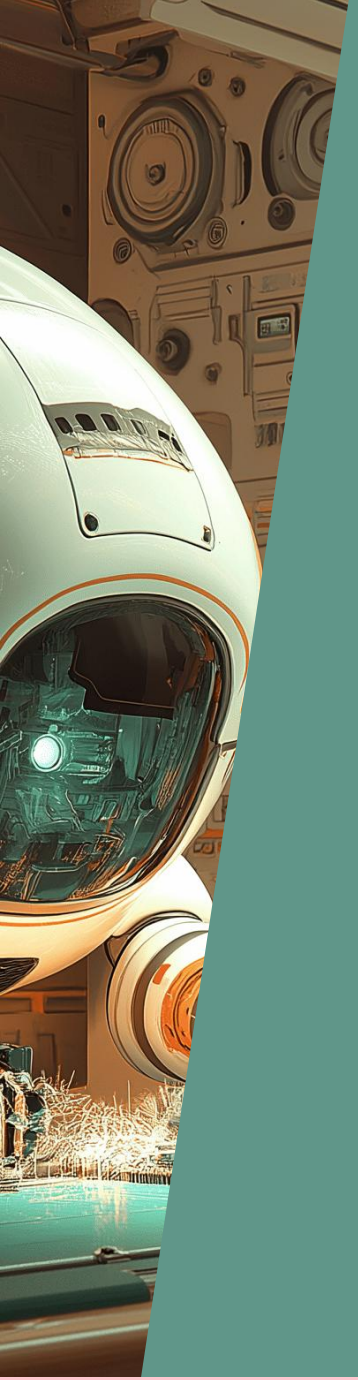


```java
@Test
void shouldProcessAsynchronously_Flaky_Slow() throws Exception {
    // GIVEN
    var processor = new AsyncProcessor();

    // WHEN
    processor.processAsync();

    // THEN: 1s suffit largement...
    Thread.sleep(1000);
    assertTrue(processor.hasSucceeded(),
            "Processing should have succeeded.");
}
```

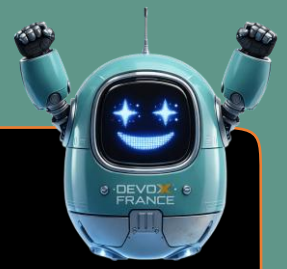
✔ shouldProcessAsynchronously_Flaky_Slow() 1 sec 14 ms



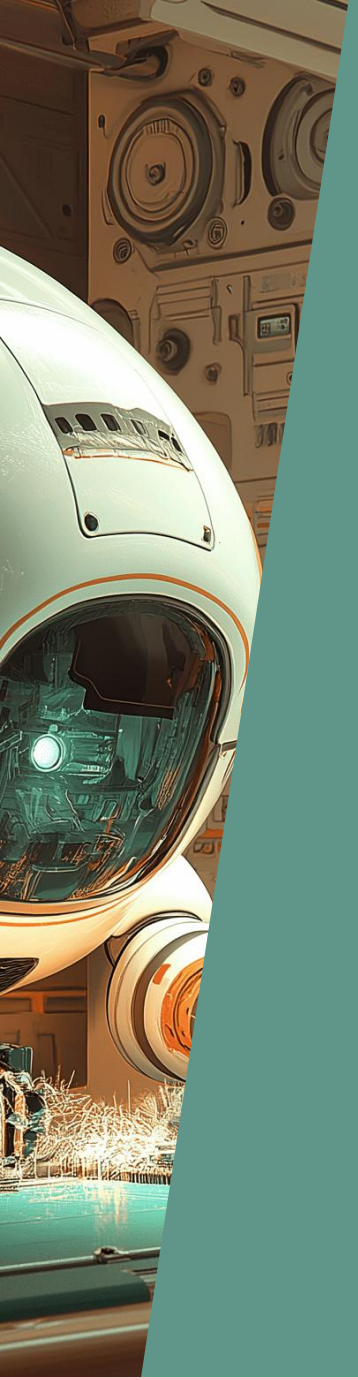
```java
@Test
void shouldProcessAsynchronously_Sleep_Correctly() throws Exception {
    // GIVEN
    var processor = new AsyncProcessor();

    // WHEN
    processor.processAsync();

    // THEN: en boucle, avec des pauses de 10ms, max 10s
    var start = System.currentTimeMillis();
    while (!processor.hasSucceeded()
            && (System.currentTimeMillis() - start) <
                    (Duration.ofSeconds(10).toMillis())) {
        Thread.sleep(10);
    }
}
```



AWAITILITY

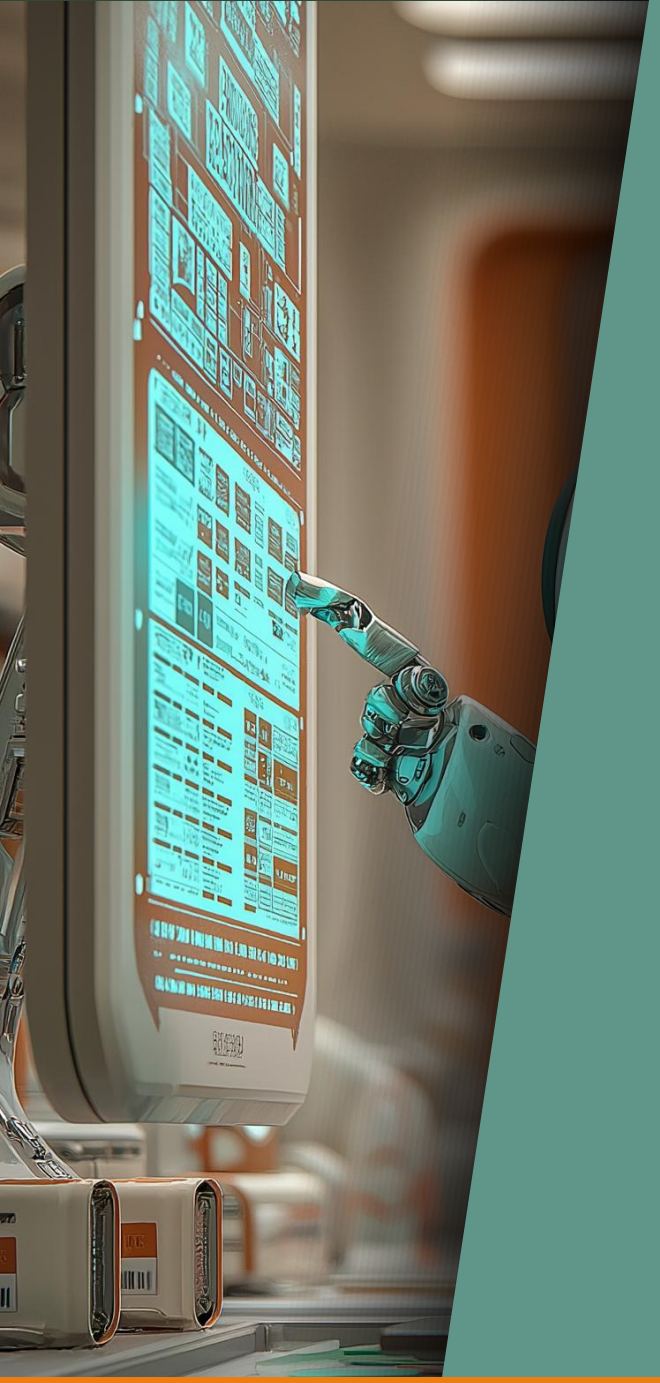



```
@Test
void shouldProcessAsynchronously_Robust() {
    // GIVEN
    AsyncProcessor processor = new AsyncProcessor();

    // WHEN
    processor.processAsync();

    // THEN: Awaitility pour attendre
    // que la condition soit satisfaite (jusqu'à 10 s)
    await()
        .atMost(Duration.ofSeconds(10)) // Max wait
        .pollInterval(Duration.ofMillis(10)) // Poll
        .until(processor::hasSucceeded); // Condition
}
```



AWAITILITY X ASSERTJ

```
@Test
void shouldProcessAsynchronously_Robust_With_AssertJ() {
    // GIVEN
    AsyncProcessor processor = new AsyncProcessor();

    // WHEN
    processor.processAsync();

    // THEN: on attend et on vérifie le contenu de la liste
    await()
        .atMost(Duration.ofSeconds(5)) // Max wait
        .pollInterval(Duration.ofMillis(10)) // Poll
        .untilAsserted(() -> {
            assertTrue(processor.hasSucceeded());
            assertThat(processor.getResult())
                .hasSize(2)
                .extracting(User::name).containsExactlyInAnyOrder("Alice", "Bob");
        });
}
```



Mauvaise Habitude #6

LA GESTION DES ÉTATS STATIQUES

```java
@Test
void old_ui_should_display_old_user() {
    var ui = new UserInterface();

    var result = ui.display("Aldrik");

    assertEquals("Aldrik", result);
}

@Test
void new_ui_should_display_new_user() {
    FeatureFlags.enable("new-ui");
    var ui = new UserInterface();

    var result = ui.display("Aldrik");

    assertEquals("ALDRIK", result);
}
```

✓ UserInterfaceTest (io.chadi.guidedutesteur.statics)	32 ms
✓ old_ui_should_display_old_user()	28 ms
✓ new_ui_should_display_new_user()	4 ms

```
[ERROR] Failures:
[ERROR]   UserInterfaceTest.old_ui_should_display_old_user:20 expected: <Aldrik> but was: <ALDRIK>
```

```
@AfterEach
void reset() {
    FeatureFlags.reset();
}

@Test
void old_ui_should_display_old_user() {
    var ui = new UserInterface();

    var result = ui.display("Aldrik");

    assertEquals("Aldrik", result);
}

@Test
void new_ui_should_display_new_user() {
    FeatureFlags.enable("new-ui");
    var ui = new UserInterface();

    var result = ui.display("Aldrik");

    assertEquals("ALDRIK", result);
}
```

```java
@TestMethodOrder(MethodOrderer.Random.class)
@Execution(ExecutionMode.CONCURRENT)
class UserInterfaceTest {
    @AfterEach
    void reset() {
        FeatureFlags.reset();
    }

    @RepeatedTest(100)
    void old_ui_should_display_old_user() {
        var ui = new UserInterface();

        var result = ui.display("Aldrik");

        assertEquals("Aldrik", result);
    }

    @RepeatedTest(100)
    void new_ui_should_display_new_user() {
        FeatureFlags.enable("new-ui");
        var ui = new UserInterface();

        var result = ui.display("Aldrik");

        assertEquals("ALDRIK", result);
    }
}
```

UserInterfaceTest (io.chadi.guidedutesteur.sta
- old_ui_should_display_old_user()
- new_ui_should_display_new_user()

EVITER LES STATIQUES !

- Évitez les variables statiques afin que les tests ne se polluent pas entre eux :
  - Un conteneur d'injection de dépendances
  - Un strategy pattern dès le plus haut niveau
  - Des variables d'environnement / propriétés
- Evitez les mockStatic lorsque c'est possible :
  - Peuvent causer des problèmes de compatibilités lors d'upgrade de JDK

Conclusion

Testez le comportement métier : Utilisez votre esprit critique et ne vous basez pas uniquement sur les métriques ou l'IA.

Investissez dans l'outillage : **Awaitility**, **AssertJ** ou **Testcontainers** rendent la lecture des tests et de leurs résultats plus agréable.

Supprimez le bruit : Éliminez tout ce qui pourrait rendre les tests aléatoires.

Chadi EL MASRI

Ingénieur chez

MUREX

https://tinyurl.com/guide-du-testeur